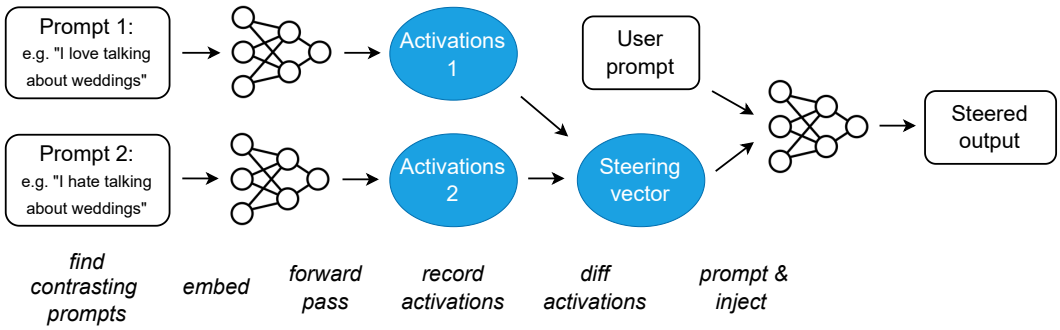


Figure 1: Schematic of the Activation Addition (**ActAdd**) method. ⬭ = natural language text; ● = vectors of activations just before a specified layer. In this example, the output is heavily biased towards discussing weddings, regardless of the topic of the user prompt. (See Algorithm 1 for the method's parameters: intervention strength, intervention layer, and sequence alignment.)

3 How Activation Addition works

We use decoder-only Transformer neural networks (Vaswani et al. 2017). The LLMs in this work contain a stack of Transformer layers, each consisting of multi-head attention (MHA) and a feedforward network (FFN). We focus on its "residual streams" (Elhage et al. 2021), the sequences $(\mathbf{x}_0, ..., \mathbf{x}_n)$ of intermediate activation vectors processed by each layer. ActAdd manipulates the residual stream values $\mathbf{h}^l$ input to layer l. Each layer performs MHA and FFN computations on $\mathbf{x}_i$, adding $\mathbf{x}_{i+1}$ to the stream. The final vector $\mathbf{x}_n$ in the stream can then be decoded into the next-token prediction. At inference time, the residual stream is initialized $\mathbf{h}^1$ with the embedding of the tokenized prompt.

Activation addition. Our method takes a pair of natural-language prompts (p_+, p_-), where p_+ represents the property we wish output text to emphasise (e.g. love) and p_- represents its opposite (e.g. hate or indifference). $\mathbf{h}^l_+$ is the activation vector for the prompt p_+ at layer l. The difference $\mathbf{h}^l_+ - \mathbf{h}^l_-$ is a new activation vector which (intuitively) captures the difference between a prompt with the target property, and a prompt without it. The steering vector is computed before inference time.

Algorithm 1 ActAdd, optimization-free activation addition

Input: (p_+, p_-) = steering prompt pair, tokenized
p^* = user prompt
l = target layer
c = injection coefficient
a = sequence position to align $\mathbf{h}_A$ and $\mathbf{h}_{p^*}$
M = pretrained language model
Output: S = steered output

$(p'_+, p'_-) \leftarrow$ `pad_right_same_token_len`(p_+, p_-)
$\mathbf{h}^l_+ \leftarrow M$`.forward(`$p'_+$`).activations[`$l$`]`
$\mathbf{h}^l_- \leftarrow M$`.forward(`$p'_-$`).activations[`$l$`]`
$\mathbf{h}^l_A \leftarrow \mathbf{h}^l_+ - \mathbf{h}^l_-$
$\mathbf{h}^l \leftarrow M$`.forward(`$p^*$`).activations[`$l$`]`
$S \leftarrow M$`.continue_forward(`$c\,\mathbf{h}^l_A + \mathbf{h}^l \mathbin{@} a$`)`

To obtain a steering vector, we perform a forward pass on each prompt, record the activations at the given location in each pass, take the difference $\mathbf{h}^l_+ - \mathbf{h}^l_-$, and then finally rescale this difference in activations by an 'injection coefficient' c. To steer, we add the resulting activation vector to the input of layer l and allow the forward pass to continue, and so obtain our steered output.[1] c represents the

[1] See Appendix C for implementation details.

intervention strength, since it multiplies the steering vector's contribution to the residual stream.[2] We perform hyperparameter tuning to select c and also the injection layer l. As expected from past work (Subramani et al. 2022; Mini et al. 2023), intervening at the middle layers is most effective.

Algorithm 1 and Figure 1 depict the resulting ActAdd method. In the appendix, Figure 6 illustrates a figurative example of steering a model with ActAdd if that model had one-dimensional residual streams (rather than e.g. GPT-2-XL's 1600 dimensions). A runnable notebook can be found at tinyurl.com/actadd.

We test whether 1) steering vectors are effective at eliciting the desired behavioral shift, and 2) whether they preserve the general capabilities of the model. We run perplexity-based experiments on GPT-2-XL (1.5B parameters, Radford et al. 2019). We then run toxicity and sentiment experiments on OPT (Zhang et al. 2022b) and LLaMA-3 (Meta 2024).

4 Results: Activation Addition works

A summary of all experiments can be found in Table 5.[3]

4.1 ActAdd intuitively modifies next-token probabilities

We consider the OpenWebText corpus (Peterson et al. 2018). Our running example is the "wedding" topic vector produced by setting $p_+ = \texttt{weddings}$, $p_- = \texttt{`\ \ '}$, $l = 16$, $c = 1$.

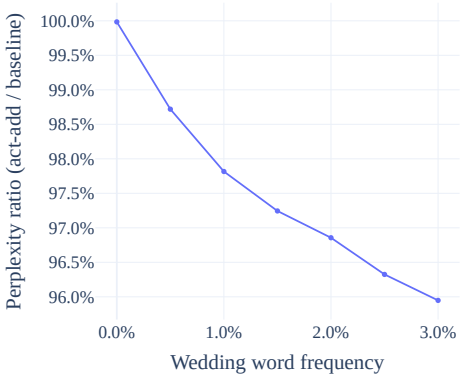
4.1.1 ActAdd reduces perplexity on a target topic

For each document $d_i \in D$ in OpenWebText (Peterson et al. 2018), we first calculate the frequency of wedding-related words.[4] If a document contains one of these words, the document is considered wedding-related. We randomly sample 300k documents, half of which are wedding-related.

We split the documents into sentences and measure GPT-2-XL's perplexity on both the wedding-related and wedding-unrelated sentences. If the model is effectively steered to generate wedding-related text, it should assign that text higher probability (and thus achieve lower perplexity). For more details, see Appendix C.1.

Figure 2 shows the ActAdd perplexity relative to the unmodified model. In sentences where the injected topic (weddings) is more relevant, ActAdd's perplexity is lower and predictive performance increases.

Figure 2: The perplexity ratio compares the relative predictive performance of ActAdd and an unmodified model. Lower is better. Adding the wedding steering vector improves performance on wedding-related text while preserving performance on unrelated text.



4.1.2 ActAdd's impact on token probabilities

To test if the intervention is affecting relevant tokens or reducing perplexity in some spurious way, we observe the shift in the distribution of token log probabilities. We do this by randomly sampling 500 documents from the above OpenWebText sample and recording the log-probabilities assigned by the baseline and steered models. This results in a dataset of about 500k tokens, of which 29k are unique. We then group by token, filter for tokens with >20 instances in the dataset, and calculate the mean perplexity difference between the ActAdd and baseline models. By displaying these as a Q-Q plot (Gnanadesikan & Wilk 1968), we can inspect outlier shifts in token probability.

[2] It's typical for the intervention strength c to have a magnitude less than 15.

[3] Code repository for our experiments: `https://zenodo.org/records/13879423`.

[4] `wedding`, `weddings`, `wed`, `marry`, `married`, `marriage`, `bride`, `groom`, and `honeymoon`.

Appendix Figure 8 shows the resulting mean log-probability difference distribution. We see that is approximately normal for the bulk of the tokens but with clearly heavy tails. The positive tail is significantly heavier than the negative tail, suggesting that one set of tokens are reliably increased in probability, with a smaller set of tokens reliably decreased to a lesser extent. Outlier tokens can be found in Appendix Table 11. *The probabilities most increased on average are primarily wedding-related.* The bottom tokens share no obvious theme and show a significantly lower absolute change in probability.

4.1.3 ActAdd steers the model to discuss weddings

At what layer are steering vectors most effective? Sweeping over GPT-2-XL injection layers for the wedding vector, we measure the average count of wedding-related words given a steering vector injected at each layer.

The intervention is already effective at the very first layer, rises in effectiveness until layer 6, and then declines. For the optimal injection site, we see >90% success in topic steering (compared to a ∼2% baseline). Figure 3 shows the results of the layer sweep.

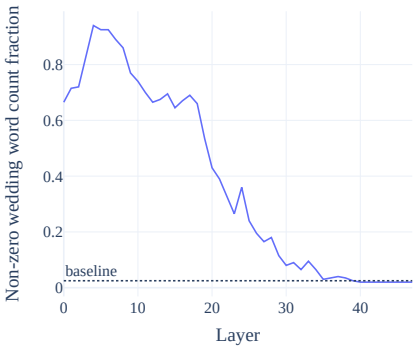


Figure 3: P(steered completion contains wedding-related words) as a function of injection layer.

4.2 ActAdd can control what the model talks about

Method. Steering vectors can elicit generations on a range of topics – not just weddings. Starting from a generic prompt, we use GPT-3.5 to score whether the generations are about a target topic. Specifically, we generate 100 completions for the unmodified model and 100 for each target single-token ActAdd intervention (each token is about a different topic). Compared to the baseline generations, we record how much *more* frequently the steered model discusses the target topic.

Results. Figure 4 records a large boost in relevance (5-20%) on all topics at injection coefficient $c = 2$ (with the exception of "art").

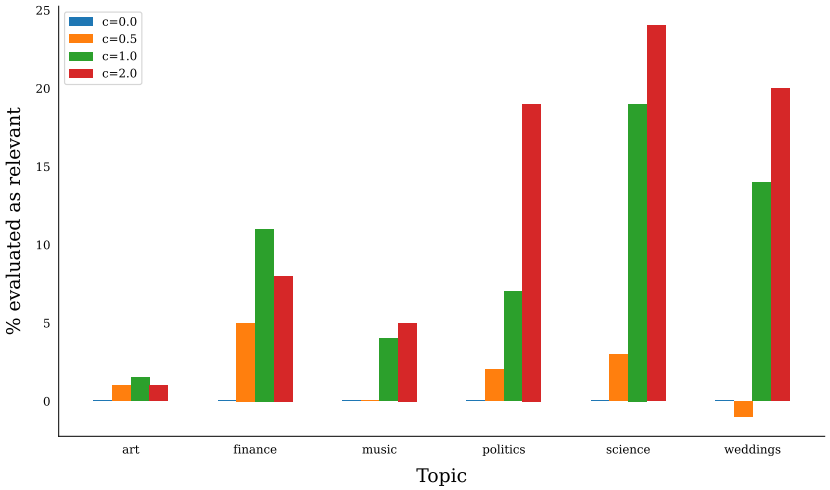


Figure 4: GPT-3.5-scored relevance of ActAdd completions on a range of generic topics.